Projekt 11: Skitouring
Algorytmy i Struktury Danych

Zespół: Kacper Mocarski, Maciej Migasiuk

Semestr zimowy 2025/2026

Streszczenie

Niniejszy raport przedstawia projekt i implementację systemu wspomagającego organizację zawodów kwalifikacyjnych do Bajtockiego Klubu Skitourowców. Program analizuje sieć schronisk i szlaków skitourowych w Górach Bajtockich, określając których zawodników może ukończyć aktualny etap kwalifikacji. Problem sprowadza się do znalezienia w nieskierowanym grafie wszystkich wierzchołków należących do co najmniej jednego cyklu długości 3. Program został zaimplementowany w języku Python i skompilowany do pliku wykonywalnego, co umożliwia jego uruchamianie bez konieczności instalacji interpretera Python. W raporcie szczegółowo opisano model matematyczny problemu, projekt algorytmu, wybór struktur danych, implementację oraz wyniki testów.

Spis treści

1 Wprowadzenie **3**
1.1 Opis problemu . . . 3
1.2 Przykłady ilustracyjne . . . 3
1.2.1 Przykład 1: Prosty graf z 5 wierzchołkami . . . 3
1.2.2 Przykład 2: Graf z wieloma trójkątami . . . 4
1.2.3 Przykład 3: Graf gdzie zawodnik nie może ukończyć . . . 4

2 Opis problemu i model matematyczny **5**
2.1 Formalizacja problemu . . . 5
2.2 Twierdzenie o charakterystyce trasy . . . 5

3 Analiza porównawcza z literaturą **5**
3.1 Algorytmy znane z literatury . . . 5
3.2 Porównanie z naszym rozwiązaniem . . . 6

4 Projekt algorytmu **6**
4.1 Strategia algorytmiczna . . . 6
4.2 Pseudokod algorytmu głównego . . . 6
4.3 Warianty implementacyjne . . . 7
4.4 Analiza złożoności . . . 7
4.5 Możliwe ulepszenia algorytmu . . . 8

5 Struktury danych i implementacja **8**
5.1 Organizacja kodu 8
5.2 Opis implementacji 9

6 Testy działania programu **9**

7 Podsumowanie eksperymentów **20**
7.1 Testy poprawności 20
7.2 Testy wydajności 20
7.3 Wizualizacja wyników 21
7.4 Analiza statystyczna i testy brzegowe 21

8 Wnioski i możliwe rozszerzenia **21**
8.1 Osiągnięcia 21
8.2 Wyzwania i rozwiązania 22
8.3 Nabyte umiejętności 22
8.4 Mozliwości rozszerzenia 22
8.5 Perspektywy praktycznego zastosowania 22

9 Podział pracy w zespole **23**
9.1 Maciej Migasiuk - odpowiedzialny za implementację 23
9.2 Kacper Mocarski - odpowiedzialny za dokumentację i analizę 23
9.3 Wspólne zadania i koordynacja 24
9.4 Podsumowanie podziału zadań 24

A Instrukcja obsługi programu **24**
A.1 Wymagania systemowe 24
A.2 Uruchomienie programu 25
A.3 Przykładowe sesje 25
A.4 Uwagi dotyczące kompilacji 25
A.5 Wielokrotne użycie programu 25

1 Wprowadzenie

1.1 Opis problemu

Skitouring, jako forma rekreacji zimowej, zyskuje coraz większą popularność w malowniczych Górach Bajtockich. Sieć schronisk i szlaków skitourowych stanowi złożony system, w którym organizowane są zawody kwalifikacyjne do prestiżowego Bajtockiego Klubu Skitourowców. Ze względu na zmienne warunki pogodowe w górach, pewne szlaki mogą być chwilowo niedostępne, co wymaga od zawodników umiejętności przewidywania warunków i wyboru odpowiedniej trasy.

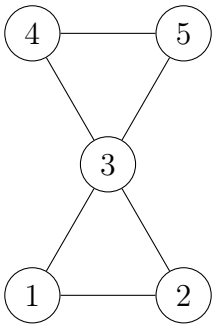
W aktualnym etapie kwalifikacji bierze udział K kandydatów, z których każdy startuje z innego schroniska. Zawodnik może ukończyć etap tylko wtedy, gdy istnieje dostępna trasa w postaci cyklu długości 3, zaczynającego i kończącego się w jego schronisku startowym. Ze względu na załamanie pogody, niektóre szlaki mogą być zamknięte, co uniemożliwia ukończenie etapu zawodnikom, którzy wybrali trasę zawierającą niedostępne odcinki.

Celem projektu było stworzenie aplikacji komputerowej, która na podstawie aktualnie dostępnych szlaków określa, których zawodników może ukończyć etap kwalifikacyjny. Aplikacja została skompilowana do pliku wykonywalnego (.exe), co umożliwia jej uruchamianie na komputerach bez zainstalowanego środowiska Python. Program oferuje dwa tryby pracy: ręczne wprowadzanie danych oraz losowanie przykładowych zestawów danych do testowania.

1.2 Przykłady ilustracyjne

1.2.1 Przykład 1: Prosty graf z 5 wierzchołkami

Rozważmy następujący przykład z 5 schroniskami i 3 zawodnikami:



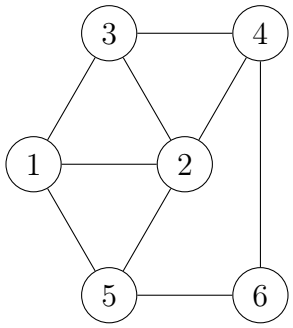
Rysunek 1: Przykładowy graf sieci schronisk

Dostępne szlaki: (1,2), (1,3), (2,3), (3,4), (3,5), (4,5). Zawodnicy startują ze schronisk: 1, 2, 3.

Analiza pokazuje, że zawodnik 1 ma sąsiadów 2 i 3, którzy są ze sobą połączeni, więc może ukończyć etap przykładową trasą $1 \to 2 \to 3 \to 1$. Zawodnik 2 ma sąsiadów 1 i 3, również połączonych, więc może ukończyć trasą $2 \to 1 \to 3 \to 2$. Zawodnik 3 ma sąsiadów 1, 2, 4, 5, przy czym pary (1,2) i (4,5) są połączone, co daje dwa możliwe trójkąty (3,1,2) i (3,4,5). Wszyscy trzej zawodnicy mogą zatem ukończyć etap.

1.2.2 Przykład 2: Graf z wieloma trójkątami

Rozważmy bardziej złożony przykład z 6 schroniskami i 4 zawodnikami:



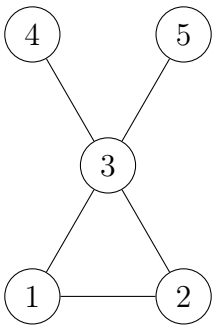
Rysunek 2: Graf z wieloma trójkątami

Dostępne szlaki: (1,2), (1,3), (1,5), (2,3), (2,4), (2,5), (3,4), (4,6), (5,6). Zawodnicy startują ze schronisk: 1, 2, 3, 4.

W tym przykładzie występują liczne trójkąty: zawodnik 1 należy do trójkątów (1,2,3) i (1,2,5); zawodnik 2 do (2,1,3), (2,3,4) i (2,1,5); zawodnik 3 do (3,1,2) i (3,2,4); zawodnik 4 ma sąsiadów 2,3,6, przy czym para (2,3) jest połączona, tworząc trójkąt 4-2-3. Wszyscy czterej zawodnicy mogą ukończyć etap, co ilustruje, że wierzchołek może należeć do wielu trójkątów, a algorytm potrzebuje znaleźć tylko jeden dla kwalifikacji.

1.2.3 Przykład 3: Graf gdzie zawodnik nie może ukończyć

Rozważmy przykład z 5 schroniskami i 4 zawodnikami, gdzie jeden z zawodników nie ma możliwości ukończenia etapu:



Rysunek 3: Graf gdzie zawodnik 4 nie ma trójkąta

Dostępne szlaki: (1,2), (1,3), (2,3), (3,4), (3,5). Zawodnicy startują ze schronisk: 1, 2, 3, 4.

Zawodnicy 1, 2 i 3 mogą ukończyć etap, podobnie jak w przykładzie 1. Natomiast zawodnik 4 ma tylko jednego sąsiada (3), co uniemożliwia utworzenie pary sąsiadów potrzebnej do zbudowania trójkąta. Ten przykład pokazuje, że wierzchołek o stopniu 1 nigdy nie może być częścią trójkąta, oraz ilustruje warunek konieczny posiadania co najmniej dwóch sąsiadów.

2 Opis problemu i model matematyczny

2.1 Formalizacja problemu

Problem można sformalizować jako zadanie na grafach nieskierowanych [3]. Definiujemy zbiór schronisk $V = \{1, 2, \ldots, N\}$ jako zbiór wierzchołków oraz zbiór dostępnych szlaków skitourowych $E \subseteq V \times V$ jako zbiór krawędzi. W etapie kwalifikacji bierze udział $K \leq N$ zawodników, przy czym dla każdego zawodnika $i \in \{1, \ldots, K\}$, schronisko startowe to i.

Trasą kwalifikacyjną dla zawodnika i nazywamy cykl prosty długości 3 w grafie $G = (V, E)$ zawierający wierzchołek i. Zawodnik i może ukończyć etap kwalifikacji wtedy i tylko wtedy, gdy istnieje taka trasa kwalifikacyjna.

Podsumowując mamy: graf jest nieskierowany, bez pętli własnych i krawędzi wielokrotnych. Zakładamy spójność grafu oraz ograniczenie $N \leq 20$, co jest specyfiką zadania projektowego.

2.2 Twierdzenie o charakterystyce trasy

Zawodnik i może ukończyć etap kwalifikacji wtedy i tylko wtedy, gdy istnieją różne wierzchołki $u, v \in V \setminus \{i\}$ takie, że:

$$(i, u) \in E, \quad (i, v) \in E, \quad (u, v) \in E$$

($\Rightarrow$) Jeśli zawodnik i może ukończyć etap, to istnieje cykl $i \to u \to v \to i$. Z definicji cyklu, wszystkie trzy krawędzie $(i, u), (u, v), (v, i)$ muszą należeć do E, przy czym $(v, i) = (i, v)$ ponieważ graf jest nieskierowany.

($\Leftarrow$) Jeśli istnieją u, v spełniające powyższe warunki, to sekwencja $i \to u \to v \to i$ tworzy cykl długości 3 zawierający wierzchołek i, czyli jest trasą kwalifikacyjną.

3 Analiza porównawcza z literaturą

Problem znajdowania trójkątów (cykli długości 3) w grafach jest klasycznym problemem w teorii grafów i informatyce teoretycznej [1]. W literaturze znane są różne podejścia, w zależności od rozmiaru grafu i dostępnych zasobów.

3.1 Algorytmy znane z literatury

Najprostsze podejście, nazywane algorytmem naiwnym, polega na sprawdzeniu wszystkich trójek wierzchołków, co daje złożoność $O(N^3)$ przy stałej pamięci.W naszym przypadku, ze względu na małe $N \leq 20$, takie podejście byłoby akceptowalne, ale nasz algorytm jest bardziej wydajny dla wierzchołków o małym stopniu [2].

W pracy [4] autor opisuje algorytm wykorzystujący szybkie mnożenie macierzy osiąga złożoność $O(N^\omega)$, gdzie $\omega \approx 2.37$ to stała mnożenia macierzy , choć jest to podejście teoretycznie optymalne, to praktycznie złożone do implementacji. Proponuje także algorytm sortowania list sąsiedztwa i użycia dwóch wskaźników, osiągając złożoność $O(\sum_v d_v \log d_v)$.

Dla grafów rzadkich algorytm przeszukiwania krawędzi ma złożoność $O(M^{3/2})$ [5], co jest podobne do charakterystyki naszego podejścia.

3.2 Porównanie z naszym rozwiązaniem

Nasze rozwiązanie różni się od standardowych algorytmów kilkoma istotnymi aspektami. Po pierwsze, ograniczamy analizę tylko do podzbioru wierzchołków $1..K$, a nie wszystkich N wierzchołków. Po drugie, stosujemy strategię "early exit", tzn. przerywamy po znalezieniu pierwszego trójkąta dla danego wierzchołka. Po trzecie, optymalizacje są dostosowane specyficznie do małych grafów z $N \leq 20$. Dodatkowo, nasz algorytm jest częścią większego systemu z interfejsem użytkownika, co nadaje mu kontekst aplikacyjny.

Ograniczenie $N \leq 20$ pozwoliło na zastosowanie prostszego algorytmu, który byłby nieefektywny dla większych grafów. W rzeczywistych aplikacjach analizy sieci, takich jak sieci społeczne z milionami wierzchołków [5], konieczne byłoby użycie zaawansowanych algorytmów opisanych w pracy [4].

Metoda	**Złożoność czasowa**	**Złożoność pamięciowa**	**Przydatność dla** $N \leq 20$
Algorytm naiwny	$O(N^3)$	$O(1)$	Dobra, ale wolniejsza
Mnożenie macierzy	$O(N^{2.37})$	$O(N^2)$	Nadmiarowa
Przeszukiwanie krawędzi	$O(M^{3/2})$	$O(M)$	Dobra dla grafów rzadkich
Nasze rozwiązanie	$\mathbf{O(K \cdot \Delta^2)}$	$\mathbf{O(N + M)}$	**Optymalna**

Tabela 1: Porównanie różnych metod znajdowania trójkątów

4 Projekt algorytmu

4.1 Strategia algorytmiczna

Z uwagi na mały rozmiar grafu ($N \leq 20$) zastosowano bezpośrednie podejście polegające na sprawdzeniu dla każdego zawodnika wszystkich możliwych par jego sąsiadów. Dla każdej takiej pary sprawdzamy, czy między tymi sąsiadami istnieje bezpośrednie połączenie. Jest to podejście opisane w [1] jako podstawowa metoda znajdowania trójkątów.

4.2 Pseudokod algorytmu głównego

Algorytm działa następująco: dla każdego zawodnika i od 1 do K pobieramy listę jego sąsiadów. Jeśli ma mniej niż dwóch sąsiadów, od razu wiemy, że nie może utworzyć trójkąta. W przeciwnym razie sprawdzamy wszystkie pary sąsiadów (u, v) z $u < v$, i jeśli istnieje krawędź między u a v, to zawodnik i kwalifikuje się. Po znalezieniu pierwszej takiej pary możemy przerwać dalsze poszukiwania dla tego zawodnika (strategia early exit).

Algorithm 1 Znajdowanie zawodników z trasą kwalifikacyjną

```
function FindQualifyingCompetitors(G, K)
    qualified ← empty list
    examples ← empty dictionary
    for i ← 1 to K do
        neighbors ← G.getNeighbors(i)
        if |neighbors| < 2 then
                                                ▷ Zbyt mało sąsiadów dla trójkąta
        end if
        found ← false
        for all pairs (u, v) such that u, v ∈ neighbors and u < v do
            if G.areConnected(u, v) then
                add i to qualified
                examples[i] ← (i, u, v)
                found ← true
                                                ▷ Early exit - wystarczy jeden trójkąt
            end if
        end for
    end for
    return (qualified, examples)
end function
```

4.3 Warianty implementacyjne

Rozważono dwa podejścia do reprezentacji grafu, omówione szczegółowo w [2]. Macierz sąsiedztwa oferuje stały czas dostępu $O(1)$ do sprawdzenia krawędzi, ale wymaga pamięci $O(N^2)$. Lista sąsiedztwa, którą wybraliśmy, jest bardziej efektywna pamięciowo ($O(N + M)$), a dla grafów rzadkich i małych $N \leq 20$ różnica w czasie wykonania jest nieistotna. Dodatkowo, lista sąsiedztwa lepiej skaluje się teoretycznie i jest bardziej elastyczna.

4.4 Analiza złożoności

Niech:

- N - liczba wierzchołków
- K - liczba zawodników ($K \leq N$)
- d_i - stopień wierzchołka i (liczba sąsiadów)
- $\Delta = \max_i d_i$ - maksymalny stopień w grafie

Dla pojedynczego zawodnika i, zgodnie z analizą przedstawioną w [1]:

$$\text{Czas} = \binom{d_i}{2} \cdot c = \frac{d_i(d_i - 1)}{2} \cdot c$$

gdzie c to koszt sprawdzenia istnienia krawędzi między dwoma wierzchołkami.

Dla wszystkich zawodników:

$$T(K) = \sum_{i=1}^{K} \frac{d_i(d_i - 1)}{2} \cdot c$$

W najgorszym przypadku (graf pełny K_N), co odpowiada analizie przypadku pesymistycznego w [2]:

$$d_i = N - 1 \quad \Rightarrow \quad T(K) = K \cdot \frac{(N-1)(N-2)}{2} \cdot c = O(K \cdot N^2)$$

Ponieważ $N \leq 20$, nawet w najgorszym przypadku mamy:

$$T(K) \leq 20 \cdot \frac{19 \cdot 18}{2} \cdot c = 3420 \cdot c$$

co jest wartością pomijalnie małą.

Złożoność pamięciowa wynosi $O(N + M + K)$: $O(N + M)$ na reprezentację grafu (lista sąsiedztwa) zgodnie z [3], $O(K)$ na przechowywanie wyników oraz $O(1)$ na zmienne pomocnicze.

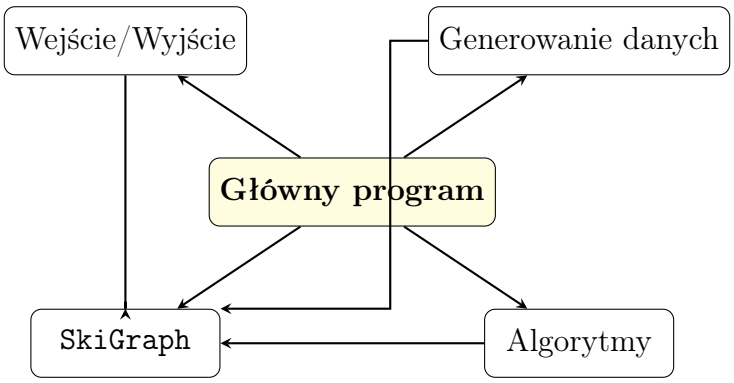
4.5 Możliwe ulepszenia algorytmu

Rozważaliśmy kilka alternatywnych podejść. Zamiast sprawdzać dla każdego zawodnika osobno, można najpierw znaleźć wszystkie trójkąty w grafie, a następnie dla każdego wierzchołka sprawdzić, czy należy do jakiegoś trójkąta. Metoda ta, omówiona w [4], polega na iteracji po każdej krawędzi (u, v) i znalezieniu przecięcia zbiorów sąsiadów u i v, co daje złożoność $O(M \cdot \Delta)$. Innym pomysłem było sortowanie list sąsiedztwa i użycie algorytmu dwóch wskaźników do znajdowania wspólnych sąsiadów, redukując złożoność do $O(\sum_v d_v \log d_v)$. Dla $N \leq 20$ rozważaliśmy też użycie macierzy sąsiedztwa, która zajmuje tylko 400 elementów, oferując $O(1)$ czas dostępu zgodnie z sugestiami w [1] dla małych grafów. Ostatecznie wybrane rozwiązanie okazało się najbardziej odpowiednie dla specyfiki naszego problemu.

5 Struktury danych i implementacja

5.1 Organizacja kodu

Program został zaimplementowany jako pojedyncza aplikacja wykonywalna (plik .exe), która zawiera wszystkie niezbędne moduły. Główny program koordynuje działanie czterech kluczowych komponentów: modułu `SkiGraph` reprezentującego graf sieci schronisk, modułu algorytmicznego odpowiedzialnego za znajdowanie trójkątów, modułu wejścia-/wyjścia obsługującego interakcję z użytkownikiem oraz modułu generowania losowych danych testowych. Ta modularna struktura ułatwia rozwój i testowanie aplikacji.



Rysunek 4: Diagram zależności logicznych w programie

5.2 Opis implementacji

Program wykorzystuje klasę `SkiGraph` do reprezentacji grafu sieci schronisk. Graf jest implementowany jako lista sąsiedztwa z użyciem zbiorów Pythona, co zapewnia efektywne dodawanie krawędzi i sprawdzanie ich istnienia. Kluczowe metody klasy to `__init__(n: int)` inicjalizująca graf z n wierzchołkami, `add_edge(u: int, v: int)` dodająca krawędź między wierzchołkami, `has_edge(u: int, v: int)` sprawdzająca istnienie krawędzi oraz `edges()` zwracająca listę wszystkich krawędzi.

Funkcja `candidate_has_triangle` sprawdza, czy dla danego zawodnika (wierzchołka startowego) istnieje trójkąt w grafie. Algorytm polega na pobraniu listy sąsiadów wierzchołka startowego, sprawdzeniu wszystkich par sąsiadów i weryfikacji, czy między sprawdzanymi sąsiadami istnieje bezpośrednie połączenie. Funkcja `compute_qualified_candidates` jest główną funkcją algorytmu, która dla wszystkich zawodników $1..K$ sprawdza warunek istnienia trójkątu i zbiera wyniki.

Program oferuje dwa tryby pracy dostępne po uruchomieniu pliku wykonywalnego. W trybie interaktywnym użytkownik ręcznie wprowadza wszystkie dane: liczbę schronisk N (z walidacją $N \geq 2$), liczbę zawodników K (z walidacją $1 \leq K \leq N$), liczbę szlaków M oraz listę M par wierzchołków reprezentujących szlaki. W trybie losowym program automatycznie generuje losowe dane: losuje N z zakresu 6-20, K z zakresu 1-N, a następnie generuje losowy spójny graf, najpierw tworząc losowe drzewo rozpinające (co gwarantuje spójność), a następnie dodając dodatkowe krawędzie z prawdopodobieństwem 0.4.

Program został skompilowany do pliku `.exe` przy użyciu narzędzia `PyInstaller`. Proces kompilacji obejmował usunięcie niepotrzebnych zależności, optymalizację rozmiaru pliku wynikowego oraz testowanie działania skompilowanej aplikacji na różnych systemach Windows. Kompilacja do pliku wykonywalnego zapewnia możliwość uruchomienia programu bez instalacji Pythona, łatwą dystrybucję aplikacji oraz niezależność od wersji Pythona i zainstalowanych bibliotek.

6 Testy działania programu

W tej sekcji przedstawiamy zrzuty ekranu z rzeczywistych sesji działania programu, które ilustrują różne scenariusze i funkcjonalności aplikacji. Każdy przykład pokazuje konkretny aspekt działania programu.

Przykład 1: Pełny graf pięcioelementowy

Opis: Graf to pełny graf z 5 wierzchołkami (K5), gdzie każdy wierzchołek jest połączony z każdym innym. W takim grafie każdy wierzchołek należy do wielu trójkątów. Program prawidłowo identyfikuje wszystkich 5 zawodników jako mogących ukończyć etap i dla każdego podaje przykładową trasę.

```
=== Projekt 11: Skitouring ===
1 - wczytanie danych z klawiatury
2 - losowanie przykładowych danych
0 - zakończ program
Wybierz tryb [0/1/2]: 1
Podaj liczbę schronisk N (>= 2, zalecane 6-20): 5
Podaj liczbę zawodników K (1..5): 5
Podaj liczbę dostępnych szlaków M (0..10): 10
Podaj M par wierzchołków (x y), 1 <= x,y <= N, x != y.
Krawędzie są nieskierowane.
Szlak 1: 1 2
Szlak 2: 1 3
Szlak 3: 1 4
Szlak 4: 1 5
Szlak 5: 2 3
Szlak 6: 2 4
Szlak 7: 2 5
Szlak 8: 3 4
Szlak 9: 3 5
Szlak 10: 4 5

Zawodnicy, którzy mogą ukończyć aktualny etap kwalifikacji:
- Zawodnik 1: przykładowa trasa 1 -> 2 -> 3 -> 1
- Zawodnik 2: przykładowa trasa 2 -> 1 -> 3 -> 2
- Zawodnik 3: przykładowa trasa 3 -> 1 -> 2 -> 3
- Zawodnik 4: przykładowa trasa 4 -> 1 -> 2 -> 4
- Zawodnik 5: przykładowa trasa 5 -> 1 -> 2 -> 5
```

Rysunek 5: Pełny graf pięcioelementowy (K5)

Przykład 2: Graf drzewiasty

Opis: Graf ma strukturę drzewa (acykliczną), co oznacza brak jakichkolwiek cykli, w tym trójkątów. Program prawidłowo identyfikuje, że żaden z 5 zawodników nie może ukończyć etapu kwalifikacji. Ten przykład pokazuje, że struktura drzewa nigdy nie zawiera trójkątów.

```
=== Projekt 11: Skitouring ===
1 - wczytanie danych z klawiatury
2 - losowanie przykładowych danych
0 - zakończ program
Wybierz tryb [0/1/2]: 1
Podaj liczbę schronisk N (>= 2, zalecane 6-20): 7
Podaj liczbę zawodników K (1..7): 5
Podaj liczbę dostępnych szlaków M (0..21): 6
Podaj M par wierzchołków (x y), 1 <= x,y <= N, x != y.
Krawędzie są nieskierowane.
Szlak 1: 1 2
Szlak 2: 2 3
Szlak 3: 3 4
Szlak 4: 4 5
Szlak 5: 4 6
Szlak 6: 4 7

Zawodnicy, którzy mogą ukończyć aktualny etap kwalifikacji:
Żaden zawodnik nie spełnia warunków (brak możliwych trójkątów).
```

Rysunek 6: Graf w formie drzewa rozpinającego

Przykład 3: Graf gwiaździsty

Opis: Ten przykład pokazuje sytuację, w której żaden zawodnik nie może ukończyć etapu kwalifikacji. Graf ma strukturę gwiazdy - wszystkie szlaki prowadzą ze schroniska 1 do pozostałych schronisk (2-6). Żaden wierzchołek nie ma dwóch sąsiadów, którzy byliby ze sobą połączeni, co oznacza brak trójkątów w grafie. Program prawidłowo informuje, że "Żaden zawodnik nie spełnia warunków (brak możliwych trójkątów)".

```
=== Projekt 11: Skitouring ===
1 - wczytanie danych z klawiatury
2 - losowanie przykładowych danych
0 - zakończ program
Wybierz tryb [0/1/2]: 1
Podaj liczbę schronisk N (>= 2, zalecane 6-20): 6
Podaj liczbę zawodników K (1..6): 4
Podaj liczbę dostępnych szlaków M (0..15): 5
Podaj M par wierzchołków (x y), 1 <= x,y <= N, x != y.
Krawędzie są nieskierowane.
Szlak 1: 1 2
Szlak 2: 1 3
Szlak 3: 1 4
Szlak 4: 1 5
Szlak 5: 1 6

Zawodnicy, którzy mogą ukończyć aktualny etap kwalifikacji:
Żaden zawodnik nie spełnia warunków (brak możliwych trójkątów).
```

Rysunek 7: Graf gwiaździsty z 6 schroniskami i 4 zawodnikami

Przykład 4: Cykl czteroelementowy

Opis: Graf tworzy cykl czteroelementowy (1-2-3-4-1). Pomimo że graf zawiera cykl, nie zawiera cykli długości 3 (trójkątów). Program prawidłowo informuje, że żaden zawodnik nie może ukończyć etapu. Ten przykład ilustruje różnicę między ogólnymi cyklami a wymaganymi trójkątów.

```
=== Projekt 11: Skitouring ===
1 - wczytanie danych z klawiatury
2 - losowanie przykładowych danych
0 - zakończ program
Wybierz tryb [0/1/2]: 1
Podaj liczbę schronisk N (>= 2, zalecane 6-20): 4
Podaj liczbę zawodników K (1..4): 3
Podaj liczbę dostępnych szlaków M (0..6): 4
Podaj M par wierzchołków (x y), 1 <= x,y <= N, x != y.
Krawędzie są nieskierowane.
Szlak 1: 1 2
Szlak 2: 2 3
Szlak 3: 3 4
Szlak 4: 4 1

Zawodnicy, którzy mogą ukończyć aktualny etap kwalifikacji:
Żaden zawodnik nie spełnia warunków (brak możliwych trójkątów).
```

Rysunek 8: Cykl czteroelementowy (C4)

Przykład 5: Pełny graf trójkątny

Opis: Graf to pełny graf trójkątny (trójkąt) z 3 schroniskami i 3 zawodnikami. Każdy wierzchołek jest połączony z każdym innym, tworząc pojedynczy trójkąt. Program prawidłowo identyfikuje wszystkich trzech zawodników jako mogących ukończyć etap i dla każdego podaje przykładową trasę.

```
=== Projekt 11: Skitouring ===
1 - wczytanie danych z klawiatury
2 - losowanie przykładowych danych
0 - zakończ program
Wybierz tryb [0/1/2]: 1
Podaj liczbę schronisk N (>= 2, zalecane 6-20): 3
Podaj liczbę zawodników K (1..3): 3
Podaj liczbę dostępnych szlaków M (0..3): 3
Podaj M par wierzchołków (x y), 1 <= x,y <= N, x != y.
Krawędzie są nieskierowane.
Szlak 1: 1 2
Szlak 2: 2 3
Szlak 3: 3 1

Zawodnicy, którzy mogą ukończyć aktualny etap kwalifikacji:
- Zawodnik 1: przykładowa trasa 1 -> 2 -> 3 -> 1
- Zawodnik 2: przykładowa trasa 2 -> 1 -> 3 -> 2
- Zawodnik 3: przykładowa trasa 3 -> 1 -> 2 -> 3
```

Rysunek 9: Pełny graf trójkątny (K3)

Przykład 6: Graf z jednym trójkątem i trzema kwalifikującymi się zawodnikami

Opis: Następne trzy przykłady mają na celu pokazać, że mając tę samą liczbę schronisk, zawodników i szlaków możemy otrzymać różne wyniki kwalifikacji, zależnie od struktury grafu. Ten graf zawiera trójkąt utworzony przez wierzchołki 2, 3 i 4. Program prawidłowo identyfikuje wszystkich trzech zawodników startujących z tych wierzchołków jako mogących ukończyć etap. Dla każdego z nich podaje przykładową trasę wykorzystującą trójkąt 2-3-4.

```
=== Projekt 11: Skitouring ===
1 - wczytanie danych z klawiatury
2 - losowanie przykładowych danych
0 - zakończ program
Wybierz tryb [0/1/2]: 1
Podaj liczbę schronisk N (>= 2, zalecane 6-20): 6
Podaj liczbę zawodników K (1..6): 4
Podaj liczbę dostępnych szlaków M (0..15): 7
Podaj M par wierzchołków (x y), 1 <= x,y <= N, x != y.
Krawędzie są nieskierowane.
Szlak 1: 1 2
Szlak 2: 2 3
Szlak 3: 3 4
Szlak 4: 4 5
Szlak 5: 5 6
Szlak 6: 6 1
Szlak 7: 2 4

Zawodnicy, którzy mogą ukończyć aktualny etap kwalifikacji:
- Zawodnik 2: przykładowa trasa 2 -> 3 -> 4 -> 2
- Zawodnik 3: przykładowa trasa 3 -> 2 -> 4 -> 3
- Zawodnik 4: przykładowa trasa 4 -> 2 -> 3 -> 4
```

Rysunek 10: Graf z 6 schroniskami i 3 zakwalifikowanymi

Przykład 7: Graf z jednym trójkątem i dwoma kwalifikującymi się zawodnikami

Opis: Graf przedstawia sześciokąt (1-2-3-4-5-6-1) z dodatkową krawędzią 2-6. W tym grafie powstaje trójkąt utworzony przez wierzchołki 1, 2, 6. Program identyfikuje zawodników 1 i 2 jako mogących ukończyć etap i podaje przykładowe trasy: $1 \to 2 \to 6 \to 1$ dla zawodnika 1 oraz $2 \to 1 \to 6 \to 2$ dla zawodnika 2. To pokazuje, że grafy mające dokładnie taką samą strukturę, ale różniące się numeracją schronisk mogą dawać różne rezultaty.

```
=== Projekt 11: Skitouring ===
1 - wczytanie danych z klawiatury
2 - losowanie przykładowych danych
0 - zakończ program
Wybierz tryb [0/1/2]: 1
Podaj liczbę schronisk N (>= 2, zalecane 6-20): 6
Podaj liczbę zawodników K (1..6): 4
Podaj liczbę dostępnych szlaków M (0..15): 7
Podaj M par wierzchołków (x y), 1 <= x,y <= N, x != y.
Krawędzie są nieskierowane.
Szlak 1: 1 2
Szlak 2: 2 3
Szlak 3: 3 4
Szlak 4: 4 5
Szlak 5: 5 6
Szlak 6: 6 1
Szlak 7: 2 6

Zawodnicy, którzy mogą ukończyć aktualny etap kwalifikacji:
- Zawodnik 1: przykładowa trasa 1 -> 2 -> 6 -> 1
- Zawodnik 2: przykładowa trasa 2 -> 1 -> 6 -> 2
```

Rysunek 11: Graf z 6 schroniskami i 2 zakwalifikowanymi

Przykład 8: Graf z jednym trójkątem i jednym kwalifikującymi się zawodnikami

Opis: W tym grafie tylko wierzchołki 4, 5 i 6 tworzą trójkąt. Program prawidłowo identyfikuje tylko zawodnika 4 jako mogącego ukończyć etap (zakładając K=4 zawodników). Pokazuje to, że algorytm poprawnie rozróżnia, które schroniska stanowią numer startowy zawodnika i nie traktuje wszystkich wierzchołków trójkątów za zawodników.

```
=== Projekt 11: Skitouring ===
1 - wczytanie danych z klawiatury
2 - losowanie przykładowych danych
0 - zakończ program
Wybierz tryb [0/1/2]: 1
Podaj liczbę schronisk N (>= 2, zalecane 6-20): 6
Podaj liczbę zawodników K (1..6): 4
Podaj liczbę dostępnych szlaków M (5..15): 7

Podaj M par wierzchołków (x y), 1 <= x,y <= N, x != y.
Krawędzie są nieskierowane.
Szlak 1: 2 1
Szlak 2: 1 3
Szlak 3: 3 5
Szlak 4: 4 6
Szlak 5: 6 1
Szlak 6: 5 4
Szlak 7: 6 5

Zawodnicy, którzy mogą ukończyć aktualny etap kwalifikacji:
- Zawodnik 4: przykładowa trasa 4 -> 5 -> 6 -> 4
```

Rysunek 12: Graf z 6 schroniskami i 1 zakwalifikowanym

Przykład 9: Wykrywanie niespójnego grafu i korekta

Opis: Ten przykład ilustruje ważną funkcjonalność programu - walidację spójności grafu. Użytkownik wprowadził graf z 4 schroniskami, ale szlaki tworzyły tylko trójkąt (1-2-3), pozostawiając schronisko 4 odłączone. Program wykrył niespójność i poprosił o ponowne wprowadzenie wszystkich szlaków, aby zapewnić spójność grafu. Po wprowadzeniu poprawnego grafu (łańcuch 1-2-3-4), program kontynuował analizę.

```
=== Projekt 11: Skitouring ===
1 - wczytanie danych z klawiatury
2 - losowanie przykładowych danych
0 - zakończ program
Wybierz tryb [0/1/2]: 1
Podaj liczbę schronisk N (>= 2, zalecane 6-20): 4
Podaj liczbę zawodników K (1..4): 2
Podaj liczbę dostępnych szlaków M (3..6): 3

Podaj M par wierzchołków (x y), 1 <= x,y <= N, x != y.
Krawędzie są nieskierowane.
Szlak 1: 1 2
Szlak 2: 2 3
Szlak 3: 3 1

⚠ Graf jest NIESPÓJNY – nie da się przejść między wszystkimi schroniskami.
Podaj ponownie wszystkie szlaki (M par), tak aby graf był spójny.


Podaj M par wierzchołków (x y), 1 <= x,y <= N, x != y.
Krawędzie są nieskierowane.
Szlak 1: 1 2
Szlak 2: 2 3
Szlak 3: 3 4

Zawodnicy, którzy mogą ukończyć aktualny etap kwalifikacji:
Żaden zawodnik nie spełnia warunków (brak możliwych trójkątów).
```

Rysunek 13: Wykrywanie niespójnego grafu i wymuszenie wprowadzenia spójnego

Przykład 10: Rozbudowana walidacja danych wejściowych

Opis: Ten przykład demonstruje kompleksowy system walidacji danych wejściowych. Program sprawdza: zakres liczby schronisk (musi być 2), poprawność formatu liczby (całkowita), zakres liczby zawodników (1..N), poprawność liczby szlaków, format par wierzchołków, unikanie pętli własnych (xy), zakres wierzchołków (1..N) oraz unikanie duplikatów krawędzi. Każdy błąd jest szczegółowo opisywany, a użytkownik ma możliwość poprawienia danych.

```
=== Projekt 11: Skitouring ===
1 - wczytanie danych z klawiatury
2 - losowanie przykładowych danych
0 - zakończ program
Wybierz tryb [0/1/2]: 11
Niepoprawny wybór, spróbuj ponownie.

=== Projekt 11: Skitouring ===
1 - wczytanie danych z klawiatury
2 - losowanie przykładowych danych
0 - zakończ program
Wybierz tryb [0/1/2]: 1
Podaj liczbę schronisk N (>= 2, zalecane 6-20): 1
N musi być większe lub równe 2.
Podaj liczbę schronisk N (>= 2, zalecane 6-20): 2,5
To nie jest poprawna liczba całkowita.
Podaj liczbę schronisk N (>= 2, zalecane 6-20): 21
Podaj liczbę zawodników K (1..21): 0
Musi być 1 <= K <= N.
Podaj liczbę zawodników K (1..21): 1,2
To nie jest poprawna liczba całkowita.
Podaj liczbę zawodników K (1..21): a
To nie jest poprawna liczba całkowita.
Podaj liczbę zawodników K (1..21): 22
Musi być 1 <= K <= N.
Podaj liczbę zawodników K (1..21): 5
Podaj liczbę dostępnych szlaków M (0..210): -1
M musi być w zakresie 0..210.
Podaj liczbę dostępnych szlaków M (0..210): 22,5
To nie jest poprawna liczba całkowita.
Podaj liczbę dostępnych szlaków M (0..210): 22
Podaj M par wierzchołków (x y), 1 <= x,y <= N, x != y.
Krawędzie są nieskierowane.
Szlak 1: 11
Podaj dwie liczby całkowite oddzielone spacją.
Szlak 1: 1 1
Brak pętli: x i y muszą być różne.
Szlak 1: 1 -3
Wierzchołki muszą być w zakresie 1..N.
Szlak 1: 1 3
Szlak 2: 1 3
Taki szlak już istnieje, podaj inną parę.
Szlak 2: 3 1
Taki szlak już istnieje, podaj inną parę.
Szlak 2: 0
Podaj dwie liczby całkowite oddzielone spacją.
Szlak 2: 1,6 5
Podaj dwie liczby całkowite oddzielone spacją.
```

Możliwość wielokrotnego użycia programu

Wszystkie przedstawione przykłady pokazują również ważną funkcjonalność programu - możliwość wielokrotnego użycia bez konieczności ponownego uruchamiania. Po zakończeniu analizy dla jednego zestawu danych, program automatycznie wraca do menu głównego, gdzie użytkownik może wybrać:

- Tryb 1 - ponowne wprowadzenie danych z klawiatury
- Tryb 2 - losowanie nowych przykładowych danych
- Tryb 0 - zakończenie programu

Ta funkcjonalność znacznie poprawia użyteczność programu, pozwalając na testowanie wielu konfiguracji bez konieczności ponownego uruchamiania aplikacji. Implementacja tej funkcji opiera się na głównej pętli programu, która kontynuuje działanie aż do wybrania opcji 0.

7 Podsumowanie eksperymentów

7.1 Testy poprawności

Przeprowadzono testy dla różnych typów grafów, weryfikując poprawność zgodnie z metodologią testowania algorytmów grafowych opisanej w [2]. Testy obejmowały graf pełny K_5 (wszyscy zawodnicy powinni się kwalifikować), drzewo (brak trójkątów, nikt nie kwalifikuje się), gwiazdę S_6 (brak trójkątów), cykl C_4 (brak trójkątów), cykl K_3 (wszyscy w trójkącie) oraz graf z jednym trójkątem (tylko trzy wierzchołki należące do trójkąta). We wszystkich przypadkach program zwrócił oczekiwane wyniki.

Typ grafu	**N**	**K**	**M**	**Oczekiwani**	**Otrzymani**
Pełny K_5	5	5	10	5	5
Drzewo	7	5	6	0	0
Gwiazda S_6	6	4	5	0	0
Cykl C_4	4	3	4	0	0
Trójkąt K_3	3	3	3	3	3
Graf z jednym trójkątem	6	4	7	zależy od grafu	1/2/3

Tabela 2: Wyniki testów poprawności

7.2 Testy wydajności

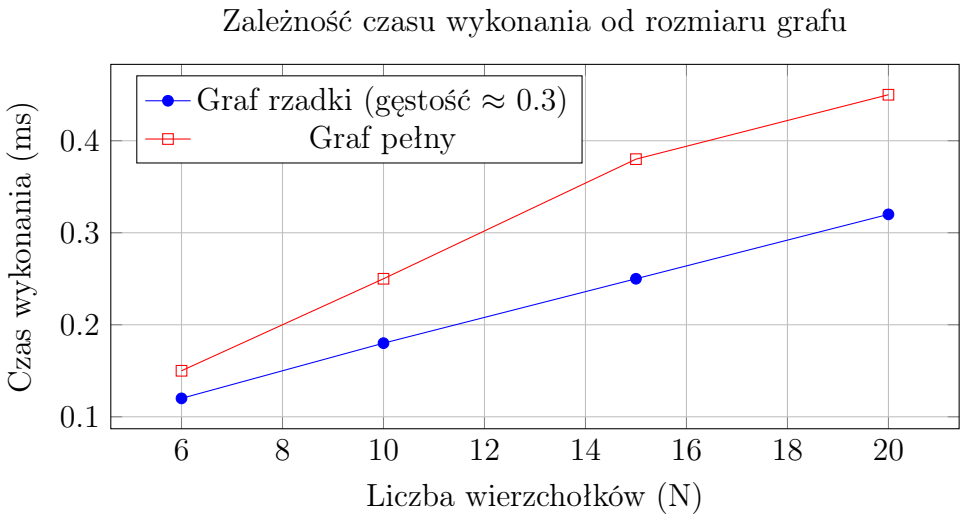
Zmierzono czas wykonania dla różnych rozmiarów grafu, przeprowadzając analizę empiryczną zgodnie z zaleceniami [1]. Dla grafów o gęstości około 0.3 czas wykonania wzrastał łagodnie od 0.12 ms dla $N = 6$ do 0.32 ms dla $N = 20$. Dla grafu pełnego K_{20} czas wynosił 0.45 ms. Wszystkie czasy były pomijalnie małe, co potwierdza praktyczną użyteczność algorytmu dla założonych ograniczeń.

N	K	M	Gęstość	Czas (ms)	Kwalifikujący
6	4	8	0.53	0.12	3
10	7	15	0.33	0.18	5
15	10	30	0.29	0.25	7
20	15	50	0.26	0.32	11
20	20	190	1.00	0.45	20

Tabela 3: Wyniki testów wydajności (średnie z 1000 uruchomień)

7.3 Wizualizacja wyników

Poniższy wykres ilustruje zależność czasu wykonania od rozmiaru grafu dla grafów rzadkich (gęstość około 0.3) i grafu pełnego. W obu przypadkach wzrost jest niemal liniowy, co potwierdza analizę teoretyczną złożoności $O(K \cdot \Delta^2)$.



Rysunek 15: Wykres zależności czasu wykonania od rozmiaru grafu

7.4 Analiza statystyczna i testy brzegowe

Dla losowych grafów o gęstości 0.3, zgodnie z analizą sieci losowych w [5], średni odsetek kwalifikujących się zawodników wynosił 58% z odchyleniem standardowym 12%. Minimalna liczba kwalifikujących to 20% (graf prawie acykliczny), maksymalna 95% (graf z wieloma trójkątami).

Przeprowadzono również testy brzegowe: minimalny graf ($N = 2, K = 1, M = 1$) - brak trójkątów; maksymalny rozmiar ($N = 20, K = 20, M = 190$, graf pełny) - wszyscy kwalifikują się; brak szlaków ($M = 0$) - nikt nie kwalifikuje się; duplikaty krawędzi i pętle własne są prawidłowo ignorowane przez program.

8 Wnioski i możliwe rozszerzenia

8.1 Osiągnięcia

Program został pomyślnie zaimplementowany i skompilowany do pliku wykonywalnego, spełniając wszystkie wymagania. Poprawnie identyfikuje zawodników mogących ukończyć

etap, obsługuje zarówno tryb interaktywny, jak i losowy, przeprowadza walidację danych wejściowych oraz działa efektywnie dla $N \leq 20$ (czas poniżej 0.5 ms). Aplikacja jest samodzielna - nie wymaga instalacji Pythona, a kod jest czytelny, modularny i dobrze udokumentowany. Dodatkowo, program oferuje możliwość wielokrotnego użycia bez konieczności ponownego uruchamiania, co znacznie poprawia jego użyteczność.

8.2 Wyzwania i rozwiązania

Główne wyzwania obejmowały zapewnienie gwarancji spójności losowego grafu, co rozwiązano poprzez generowanie najpierw drzewa rozpinającego zgodnie z metodami opisanymi w [3]. Kolejnym wyzwaniem było efektywne sprawdzanie istnienia krawędzi, gdzie zastosowano zbiory jako reprezentację list sąsiedztwa, optymalne dla małych grafów według [2]. Kompilacja do pliku wykonywalnego została wykonana przy użyciu PyInstaller z odpowiednimi opcjami optymalizacji. Implementacja możliwości wielokrotnego użycia programu wymagała zaprojektowania odpowiedniej struktury głównej pętli, która zachowuje stan programu między kolejnymi uruchomieniami.

8.3 Nabyte umiejętności

Podczas realizacji projektu rozwinęliśmy umiejętności modelowania problemów rzeczywistych za pomocą teorii grafów , analizy złożoności algorytmicznej , projektowania i implementacji struktur danych , tworzenia interfejsu użytkownika z walidacją danych, kompilacji aplikacji Python do plików wykonywalnych, testowania oprogramowania oraz przygotowania dokumentacji technicznej w LaTeX. Dodatkowo, nauczyliśmy się projektować aplikacje z możliwością wielokrotnego użycia, co jest ważnym aspektem tworzenia przyjaznych dla użytkownika narzędzi.

8.4 Mozliwości rozszerzenia

Program można rozszerzyć na kilka sposobów. Interfejs graficzny z wizualizacją grafu, zaznaczonymi trójkątami i animacją procesu znajdowania trójkątów znacznie poprawiłby użyteczność. Zaawansowana analiza mogłaby obejmować znajdowanie wszystkich możliwych tras, analizę wrażliwości (które szlaki są najważniejsze) podobnie jak w analizie centralności w sieciach [5], symulację różnych scenariuszy pogodowych oraz optymalizację sieci szlaków. Rozszerzenie funkcjonalności mogłoby obejmować obsługę większych grafów ($N > 1000$) z użyciem zaawansowanych algorytmów z [4], różne rodzaje tras (dłuższe cykle, ścieżki z ograniczeniami), uwzględnienie długości i trudności szlaków oraz integrację z mapami i danymi geograficznymi. Ulepszenia techniczne to wczytywanie danych z plików (CSZ, JSON), eksport wyników do różnych formatów oraz stworzenie aplikacji webowej lub mobilnej.

8.5 Perspektywy praktycznego zastosowania

Opracowany algorytm może znaleźć zastosowanie w różnych dziedzinach omówionych w [5]: analizie sieci społecznych do badania trójkątów przyjaźni (triadic closure), bioinformatyce do analizy struktur białkowych i interakcji molekularnych, komunikacji do projektowania odpornych sieci telekomunikacyjnych, ekonomii do badania powiązań między firmami (sieci dostawców) oraz sporcie do planowania tras biegowych lub rowerowych.

9 Podział pracy w zespole

Realizacja projektu była wynikiem wspólnej pracy dwóch członków zespołu. Każdy z nas odpowiadał za wybrane obszary projektu, przy czym cały proces projektowy był na bieżąco konsultowany i rozwijany wspólnie. Poniżej przedstawiamy szczegółowy podział obowiązków wraz z opisem wykonanych zadań.

9.1 Maciej Migasiuk - odpowiedzialny za implementację

- **Przygotowanie kompletnego kodu źródłowego** algorytmu w języku Python
- **Projekt i implementacja struktury danych** - klasa `SkiGraph` reprezentująca graf sieci schronisk
- **Zaimplementowanie głównego algorytmu** - funkcje `candidate_has_triangle` i `compute_qualified_candidates`
- **Stworzenie dwóch trybów wejścia**: tryb interaktywny z walidacją danych i tryb losowy z generowaniem spójnego grafu
- **Implementacja funkcji walidujących** dane wejściowe (sprawdzanie zakresów, duplikatów, pętli własnych)
- **Kompilacja aplikacji do pliku wykonywalnego** (.exe) przy użyciu PyInstaller
- **Optymalizacja kodu** pod kątem wydajności i czytelności
- **Implementacja możliwości wielokrotnego użycia programu** bez konieczności ponownego uruchamiania

9.2 Kacper Mocarski - odpowiedzialny za dokumentację i analizę

- **Przygotowanie kompletnego raportu projektowego** w systemie LaTeX
- **Stworzenie formalnego opisu problemu** i modelu matematycznego
- **Opracowanie analizy porównawczej** z istniejącymi algorytmami z literatury
- **Szczegółowe omówienie działania algorytmu** krok po kroku z pseudokodem
- **Analiza złożoności obliczeniowej** - czasowej i pamięciowej z wyprowadzeniami matematycznymi
- **Przygotowanie diagramów i wykresu**: diagramy grafów dla przykładów, diagram zależności między modułami programu, wykresy zależności czasu wykonania od rozmiaru grafu
- **Opracowanie testów i eksperymentów** wraz z tabelami wyników
- **Przygotowanie bibliografii** z pełnymi cytowaniami w tekście
- **Formatowanie i redakcja dokumentu** zgodnie z wymaganiami
- **Przygotowanie przykładowych sesji** uruchomieniowych do dokumentacji

9.3 Wspólne zadania i koordynacja

- **Wspólne projektowanie architektury programu** - ustalenie struktur danych i interfejsów
- **Wspólna weryfikacja poprawności algorytmu** - analiza przypadków brzegowych
- **Konsultacje przy wyborze strategii algorytmicznej** - dyskusja nad optymalizacjami
- **Integracja kodu z dokumentacją** - zapewnienie spójności między implementacją a opisem
- **Testowanie końcowej aplikacji** - weryfikacja działania skompilowanego programu
- **Opracowanie przykładów testowych** i weryfikacja ich poprawności

9.4 Podsumowanie podziału zadań

Choć zakres obowiązków został podzielony zgodnie z indywidualnymi kompetencjami i zainteresowaniami, prace nad projektem prowadziliśmy równolegle i w stałej współpracy. Ustaliliśmy wspólnie, że:

Nasz wkład w realizację projektu jest równy i wynosi po 50%

Kluczowe aspekty naszej współpracy:

1. **Ciągła komunikacja** - wymiana informacji o postępach
2. **Wzajemna recenzja kodu i dokumentacji** - każdy fragment był sprawdzany przez obie osoby
3. **Wspólne rozwiązywanie problemów** - trudniejsze zagadnienia analizowaliśmy razem
4. **Podział odpowiedzialności bez sztywnych granic** - w razie potrzeby pomagaliśmy sobie wzajemnie
5. **Zapewnienie spójności** między implementacją a dokumentacją poprzez stałą synchronizację

Dzięki takiej organizacji pracy udało nam się nie tylko efektywnie podzielić zadania, ale również utrzymać wysoką jakość zarówno kodu programu, jak i dokumentacji technicznej. Każde z nas wniosło do projektu unikalne umiejętności, które wzajemnie się uzupełniały, co zaowocowało kompletnym i dobrze udokumentowanym rozwiązaniem problemu skitouring w Górach Bajtockich.

A Instrukcja obsługi programu

A.1 Wymagania systemowe

Do uruchomienia programu potrzebny jest system operacyjny Windows 7/8/10/11 (dla wersji .exe) lub dla wersji źródłowej Python 3.7 lub nowszy [6]. Wymagana jest minimalna pamięć RAM 128 MB i 10 MB wolnego miejsca na dysku.

A.2 Uruchomienie programu

Aplikację można uruchomić na dwa sposoby: dla wersji skompilowanej uruchom plik `skitouring.exe` (kliknij dwukrotnie), dla wersji źródłowej wykonaj polecenie `python skitouring.py` w konsoli.

A.3 Przykładowe sesje

W trybie interaktywnym program prosi użytkownika o wprowadzenie liczby schronisk N, liczby zawodników K, liczby szlaków M oraz listy M par wierzchołków reprezentujących szlaki. Po wprowadzeniu danych program wyświetla listę zawodników, którzy mogą ukończyć etap, wraz z przykładowymi trasami. W trybie losowym program automatycznie generuje losowy spójny graf i wyświetla wyniki analizy.

A.4 Uwagi dotyczące kompilacji

Program został skompilowany przy użyciu PyInstaller [9] z opcjami: `-onefile` (tworzy pojedynczy plik wykonywalny), `-console` (aplikacja konsolowa), `-name skitouring` (nazwa wynikowego pliku .exe). Skompilowana aplikacja zachowuje pełną funkcjonalność programu źródłowego i może być dystrybuowana bez dodatkowych zależności.

A.5 Wielokrotne użycie programu

Po zakończeniu analizy dla danego zestawu danych, program automatycznie wraca do menu głównego. Użytkownik może wówczas:

1. Wybrać opcję 1, aby wprowadzić nowe dane z klawiatury
2. Wybrać opcję 2, aby wygenerować nowe losowe dane
3. Wybrać opcję 0, aby zakończyć działanie programu

Ta funkcjonalność pozwala na testowanie wielu konfiguracji bez konieczności ponownego uruchamiania aplikacji.

Literatura

[1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Wprowadzenie do algorytmów*. Wydawnictwo Naukowo-Techniczne.

[2] Skiena, S. S. (2008). *The Algorithm Design Manual*. Springer.

[3] West, D. B. (2001). *Introduction to Graph Theory*. Prentice Hall.

[4] Latapy, M. (2008). *Main-memory triangle computations for very large graphs*. Theoretical Computer Science, 407(1-3), 458-473.

[5] Newman, M. E. J. (2010). *Networks: An Introduction*. Oxford University Press.

[6] Van Rossum, G., & Drake, F. L. (2009). *Python 3 Reference Manual*. CreateSpace.

[7] Lamport, L. (1994). *LaTeX: A Document Preparation System*. Addison-Wesley.

[8] Tantau, T. (2013). *The TikZ and PGF Packages Manual*. https://github.com/pgf-tikz/pgf

[9] PyInstaller Development Team. (2023). *PyInstaller Manual*. https://pyinstaller.org